Integration Testing I: Strategies, Reviews, and APIs

SE 413/513 Software Testing · Demo: demos/integration_testing

Dr. Kamrul Hasan, Grand Valley State University

Delivery Plan

Introduction, Unit Testing, Test Case Design, and Test Driven Development (TDD)

Data-Flow Testing, Coverage-Based Testing, Functional Testing

Integration Testing, System Testing, and Acceptance Testing

Continuous Testing, Introduction to Advanc Testing including AI-Assisted Testing

Project (Learning by Doing)

Week no.	Week of	Topic
1	Aug. 31	Testing Foundations
2	Sept. 7	Unit Testing
3	Sept. 14	Test Case Design
4	Sept. 21	TDD and Test Doubles
5	Sept. 28	Control-Flow Testing
6	Oct. 5	Data-Flow Testing and Coverage Concepts
7	Oct. 12	Coverage-Based Testing
8	Oct. 19	Functional Testing
9	Oct. 26	Integration Testing
10	Nov. 2	System Testing
11	Nov. 9	Acceptance Testing
12	Nov. 16	Project / Testing Workshop
13	Nov. 23	Continuous Testing
14	Nov. 30	Advanced Testing, Emerging Techniques, and AI-Assisted Testing
15	Dec. 7	Project Presentation
16	Dec. 14	Exam & Final Evaluation Week

Delivery Plan

Introduction, Unit Testing, Test Case Design, and Test Driven Development (TDD)

Data-Flow Testing, Coverage-Based Testing, & Functional Testing

Integration Testing, System Testing, and Acceptance Testing

Continuous Testing, Introduction to Advanc Testing including AI-Assisted Testing

Project (Learning by Doing)

Week no.	Week of	Topic
1	Aug. 31	Testing Foundations
2	Sept. 7	Unit Testing
3	Sept. 14	Test Case Design
4	Sept. 21	TDD and Test Doubles
5	Sept. 28	Control-Flow Testing
6	Oct. 5	Data-Flow Testing and Coverage Concepts
7	Oct. 12	Coverage-Based Testing
8	Oct. 19	Functional Testing
9	Oct. 26	Integration Testing
10	Nov. 2	System Testing
11	Nov. 9	Acceptance Testing
12	Nov. 16	Project / Testing Workshop
13	Nov. 23	Continuous Testing
14	Nov. 30	Advanced Testing, Emerging Techniques, and AI-Assisted Testing
15	Dec. 7	Project Presentation
16	Dec. 14	Exam & Final Evaluation Week

Learning objectives

1

Explain

Explain which defects live between components, and why unit tests with mocks cannot see them.

2

Plan

Choose an integration strategy, list a system's integration points, and turn design-review findings into test scenarios.

3

Test an API

Test an HTTP API's status codes, formats, errors, and authentication, top-down with a stub below it.

Reading: Week 9, Sections 1-7. **Demo folder:** `demo-testing-suite/demos/integration_testing`

Plan for today

- Case study: Mars Climate Orbiter, 1999
- What is Integration Testing: Student Registration Example
- Detailed Integration Testing: Through a Practical Shop Application

Case study: Mars Climate Orbiter, 1999



The spacecraft was lost arriving at Mars on September 23, 1999.

The investigation board traced the root cause to an interface: a ground software file reported thruster performance in **pound-force seconds**, while the navigation software that consumed it expected **newton-seconds**, as the project's Software Interface Specification required.

Each component did what its developers intended. The defect lived in the connection between them.

Same class of defect as this week's "12.50" vs. 1250.

Integration Test Example: Student Registration System

1. Components and Interaction



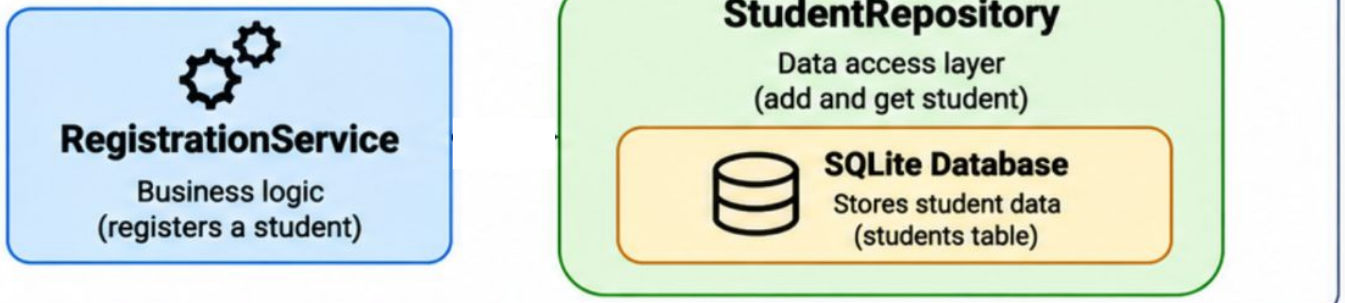
2. Application Code (registration.py)

```python
class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

Integration Test Example: Student Registration System

1. Components and Interaction



2. Application Code (registration.py)

```python
import sqlite3

class StudentRepository:
    def __init__(self, db):
        self.db = db

    def add_student(self, student_id, name):
        self.db.execute("INSERT INTO students VALUES (?, ?)",
                        (student_id, name))
        self.db.commit()

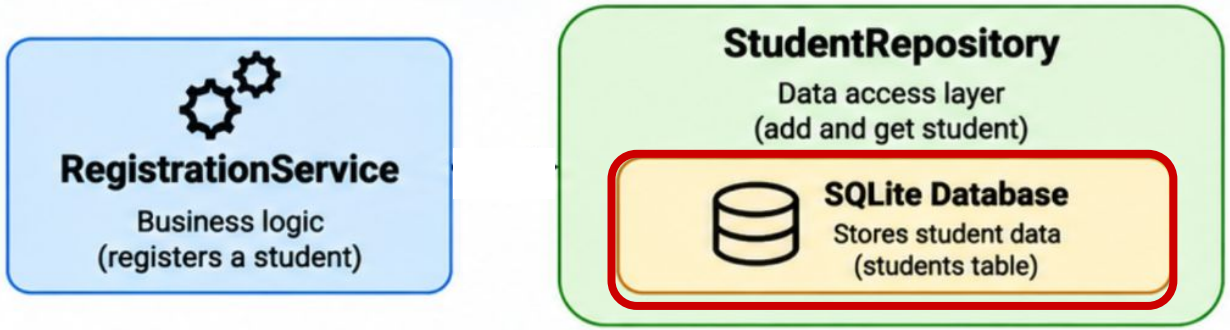
    def get_student(self, student_id):
        return self.db.execute(
            "SELECT * FROM students WHERE id = ?", (student_id,)
        ).fetchone()

class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

Integration Test Example: Student Registration System

1. Components and Interaction



2. Application Code (registration.py)

```python
import sqlite3

class StudentRepository:
    def __init__(self, db):
        self.db = db

    def add_student(self, student_id, name):
        self.db.execute("INSERT INTO students VALUES (?, ?)",
                        (student_id, name))
        self.db.commit()

    def get_student(self, student_id):
        return self.db.execute(
            "SELECT * FROM students WHERE id = ?", (student_id,)
        ).fetchone()

class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

3. Integration Test (test_registration.py)

```python
import sqlite3
import pytest
from registration import StudentRepository, RegistrationService

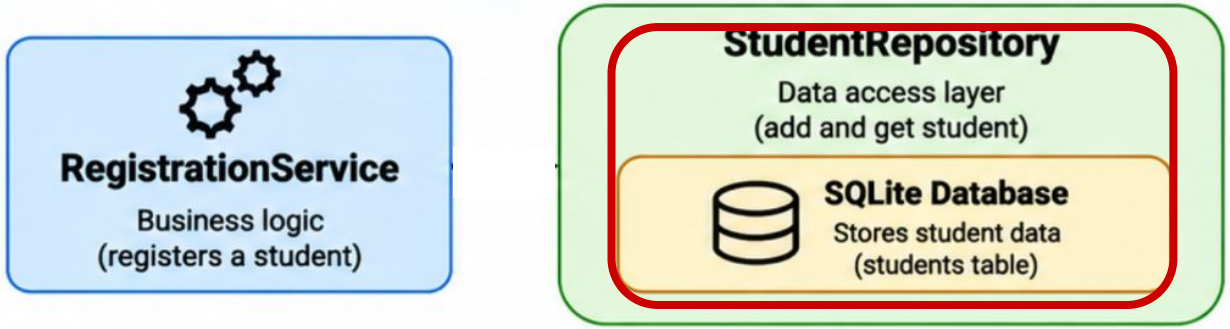
@pytlts.mark.integration
def test_student_registration():
    # Arrange: set up a test database
    db = sqlite3.connect(":memory:");
    db.execute("""
        CREATE TABLE students (
            id INTEGER PRIMARY KEY,
            name TEXT
        )
    """)
    repository = StudentRepository(db)
    service = RegistrationService(repository)
    # Act: register a student
    result = service.register(101, "John")
    # Assert: verify the result
    assert result == (101, "John")
    db.close()
```

Integration Test Flow

1. Set up a real (in-memory) SQLite database

Integration Test Example: Student Registration System

1. Components and Interaction



2. Application Code (registration.py)

```python
import sqlite3

class StudentRepository:
    def __init__(self, db):
        self.db = db

    def add_student(self, student_id, name):
        self.db.execute("INSERT INTO students VALUES (?, ?)",
                        (student_id, name))
        self.db.commit()

    def get_student(self, student_id):
        return self.db.execute(
            "SELECT * FROM students WHERE id = ?", (student_id,)
        ).fetchone()

class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

3. Integration Test (test_registration.py)

```python
import sqlite3
import pytest
from registration import StudentRepository, RegistrationService

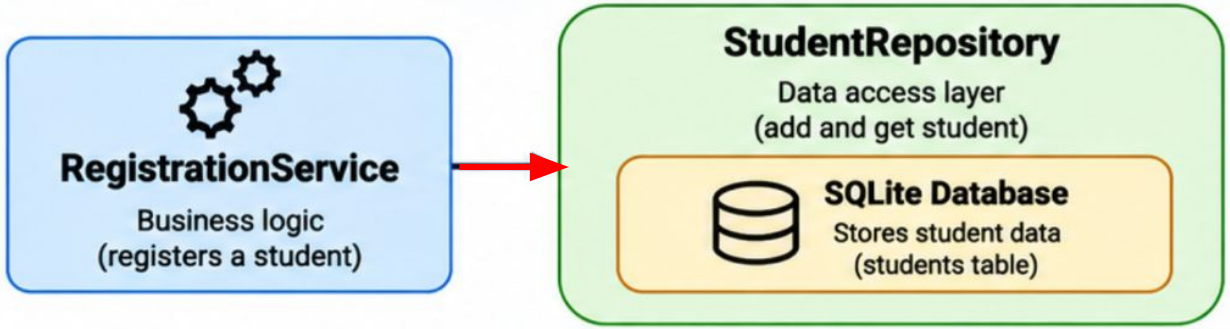
@pytlts.mark.integration
def test_student_registration():
    # Arrange: set up a test database
    db = sqlite3.connect(":memory:");
    db.execute("""
        CREATE TABLE students (
            id INTEGER PRIMARY KEY,
            name TEXT
        )
    """)
    repository = StudentRepository(db)
    service = RegistrationService(repository)
    # Act: register a student
    result = service.register(101, "John")
    # Assert: verify the result
    assert result == (101, "John")
    db.close()
```

Integration Test Flow

1. Set up a real (in-memory) SQLite database
2. Create repository (with SQLite) and service

Integration Test Example: Student Registration System

1. Components and Interaction



2. Application Code (registration.py)

```python
import sqlite3

class StudentRepository:
    def __init__(self, db):
        self.db = db

    def add_student(self, student_id, name):
        self.db.execute("INSERT INTO students VALUES (?, ?)",
                        (student_id, name))
        self.db.commit()

    def get_student(self, student_id):
        return self.db.execute(
            "SELECT * FROM students WHERE id = ?", (student_id,)
        ).fetchone()

class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

3. Integration Test (test_registration.py)

```python
import sqlite3
import pytest
from registration import StudentRepository, RegistrationService

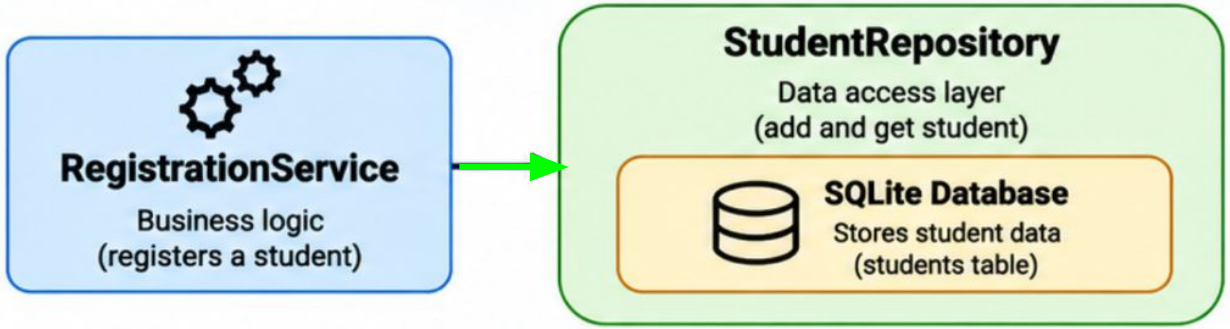
@pytlts.mark.integration
def test_student_registration():
    # Arrange: set up a test database
    db = sqlite3.connect(":memory:");
    db.execute("""
        CREATE TABLE students (
            id INTEGER PRIMARY KEY,
            name TEXT
        )
    """)
    repository = StudentRepository(db)
    service = RegistrationService(repository)
    # Act: register a student
    result = service.register(101, "John")
    # Assert: verify the result
    assert result == (101, "John")
    db.close()
```

Integration Test Flow

1. Set up a real (in-memory) SQLite database
2. Create repository (with SQLite) and service
3. Register a student (service → repository → database)

Integration Test Example: Student Registration System

1. Components and Interaction



2. Application Code (registration.py)

```python
import sqlite3

class StudentRepository:
    def __init__(self, db):
        self.db = db

    def add_student(self, student_id, name):
        self.db.execute("INSERT INTO students VALUES (?, ?)",
                        (student_id, name))
        self.db.commit()

    def get_student(self, student_id):
        return self.db.execute(
            "SELECT * FROM students WHERE id = ?", (student_id,)
        ).fetchone()

class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

3. Integration Test (test_registration.py)

```python
import sqlite3
import pytest
from registration import StudentRepository, RegistrationService

@pytlts.mark.integration
def test_student_registration():
    # Arrange: set up a test database
    db = sqlite3.connect(":memory:");
    db.execute("""
        CREATE TABLE students (
            id INTEGER PRIMARY KEY,
            name TEXT
        )
    """)

    repository = StudentRepository(db)
    service = RegistrationService(repository)
    # Act: register a student
    result = service.register(101, "John")

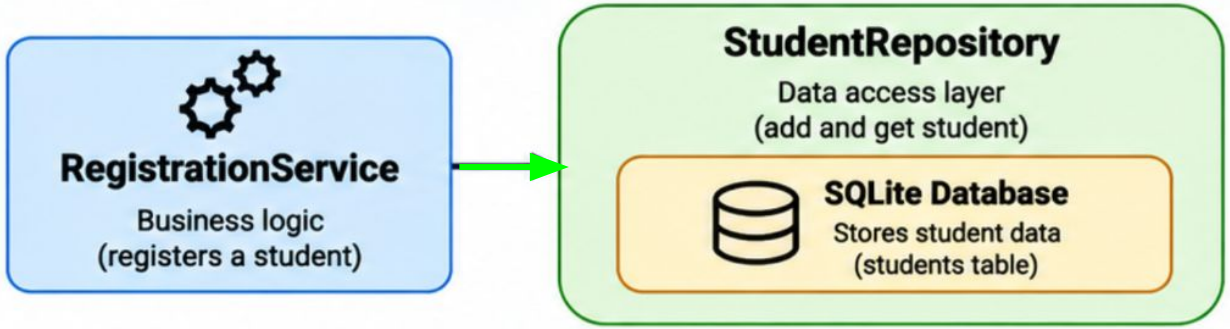
    # Assert: verify the result
    assert result == (101, "John")
    db.close()
```

Integration Test Flow

1. Set up a real (in-memory) SQLite database
2. Create repository (with SQLite) and service
3. Register a student (service → repository → database)
4. Verify the student is stored and retrieved correctly

Integration Test Example: Student Registration System

1. Components and Interaction



2. Application Code (registration.py)

```python
import sqlite3

class StudentRepository:
    def __init__(self, db):
        self.db = db

    def add_student(self, student_id, name):
        self.db.execute("INSERT INTO students VALUES (?, ?)",
                        (student_id, name))
        self.db.commit()

    def get_student(self, student_id):
        return self.db.execute(
            "SELECT * FROM students WHERE id = ?", (student_id,)
        ).fetchone()

class RegistrationService:
    def __init__(self, repository):
        self.repository = repository

    def register(self, student_id, name):
        self.repository.add_student(student_id, name)
        return self.repository.get_student(student_id)
```

3. Integration Test (test_registration.py)

```python
import sqlite3
import pytest
from registration import StudentRepository, RegistrationService

@pytlts.mark.integration
def test_student_registration():
    # Arrange: set up a test database
    db = sqlite3.connect(":memory:");
    db.execute("""
        CREATE TABLE students (
            id INTEGER PRIMARY KEY,
            name TEXT
        )
    """)

    repository = StudentRepository(db)
    service = RegistrationService(repository)
    # Act: register a student
    result = service.register(101, "John")

    # Assert: verify the result
    assert result == (101, "John")
    db.close()
```

Integration Test Flow

1. Set up a real (in-memory) SQLite database
2. Create repository (with SQLite) and service
3. Register a student (service → repository → database)
4. Verify the student is stored and retrieved correctly

4. Key Takeaway

- **Test type:** Integration test
- **Components tested together:** RegistrationService, StudentRepository (with SQLite database)
- **What it verifies:** The service correctly calls the repository, which performs database operations on SQLite, and the expected data is returned.

What is in the demo

LIVE DEMO 1
setup + look around

Setup

```
cd
demos/integration_testing_minimal/stud
ent_registration
pytest
```

```
1 passed in 0.17s
```

Run from this folder

Imports, test paths, and outputs then match the slides exactly.

Integration Strategies

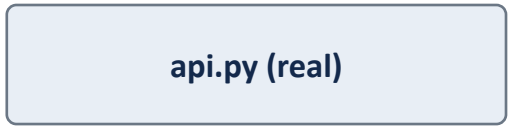
- Non-Incremental: Big-Bang Integration: In big-bang integration, all components are built (and perhaps unit tested), then connected all at once and tested as a whole.

- Incremental Integration: In incremental integration, components are added one (or a few) at a time, and the growing assembly is tested at each step. When a new failure appears, it is almost certainly in the component or connection just added. Missing components are simulated:

- Sandwich (hybrid): Work top-down and bottom-up at the same time, and meet in the middle layer. Most real teams do something like this, guided by risk: integrate first whatever is newest, least understood, or most expensive to get wrong[5, 2].

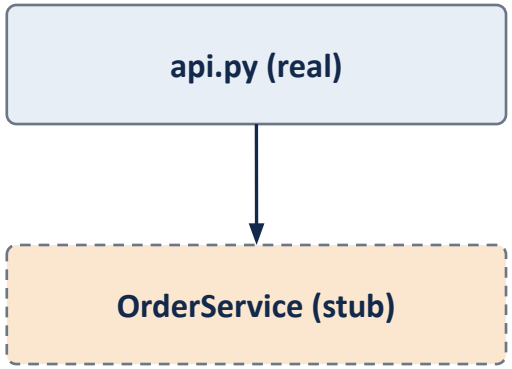
Incremental integration: stubs and drivers

Top-down: step 1



Incremental integration: stubs and drivers

Top-down: step 1

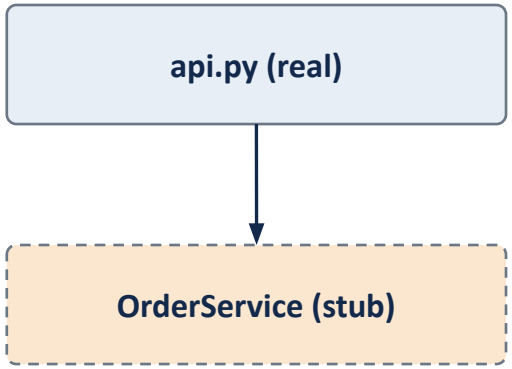


Stub

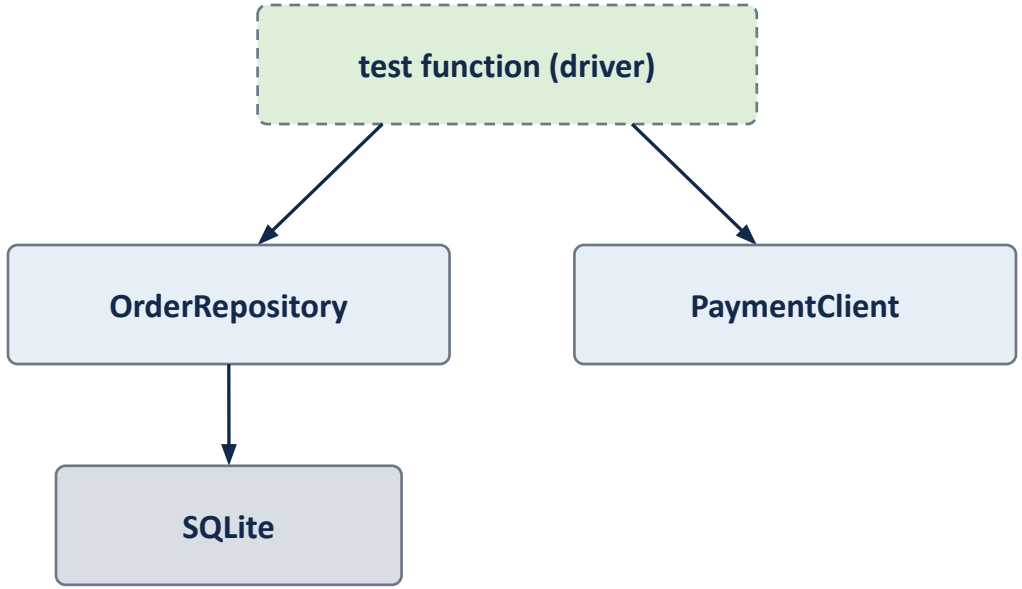
Stands in for a component below the one being tested: it is called, and returns canned answers.

Incremental integration: stubs and drivers

Top-down: step 1



Bottom-up: step 1



Stub

Stands in for a component below the one being tested: it is called, and returns canned answers.

Driver

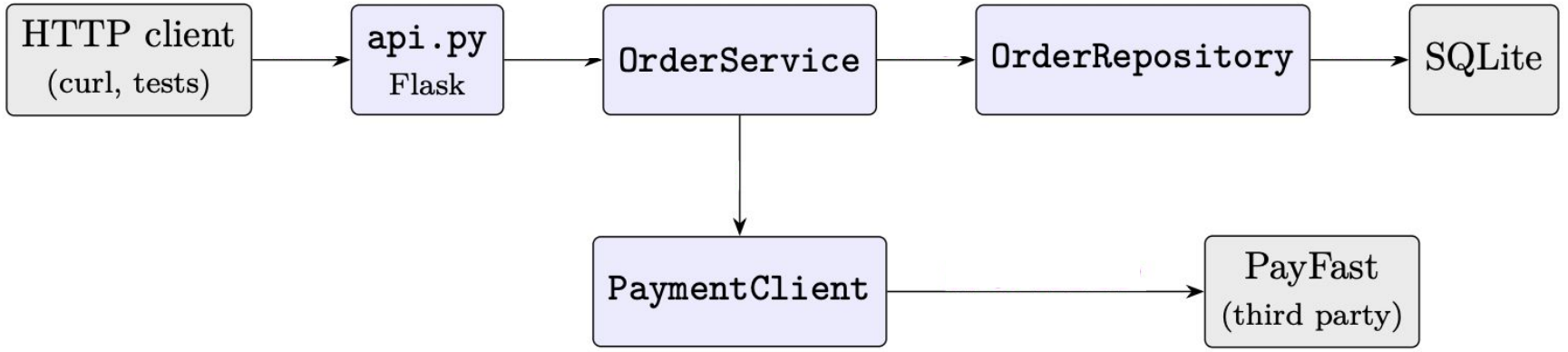
Stands in for a component above: it calls the one under test. In pytest, the test function is the driver.

Choosing a strategy

Strategy	Scaffolding	Strengths	Weaknesses
Big-bang	none	simple; fine for tiny systems	failures hard to localize; integration starts late
Top-down	stubs	early skeleton; user-visible flow tested early	many stubs; low-level risks found late
Bottom-up	drivers (tests)	risky low-level integrations tested first	whole system visible only at the end
Sandwich / risk-based	both	tests the riskiest connections first	needs planning and judgment

The demo is a sandwich. Bottom-up from the database and the payment provider, top-down from the API, meeting at OrderService. Integrate first whatever is newest, least understood, or most costly to get wrong.

Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code.

`demos/integration_testing`

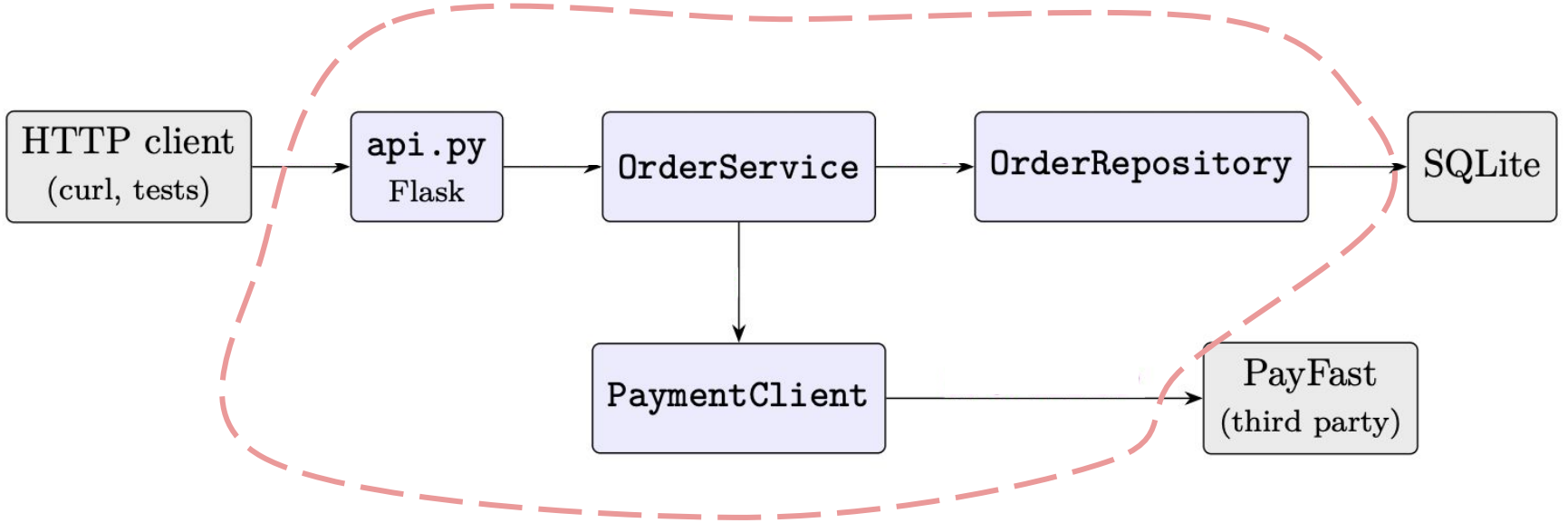
README.md

Setup

```
cd demos/integration_testing
pip install -r ../../requirements.txt
pytest
```

```
46 passed in 0.90s
```

Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend**

`demos/integration_testing`

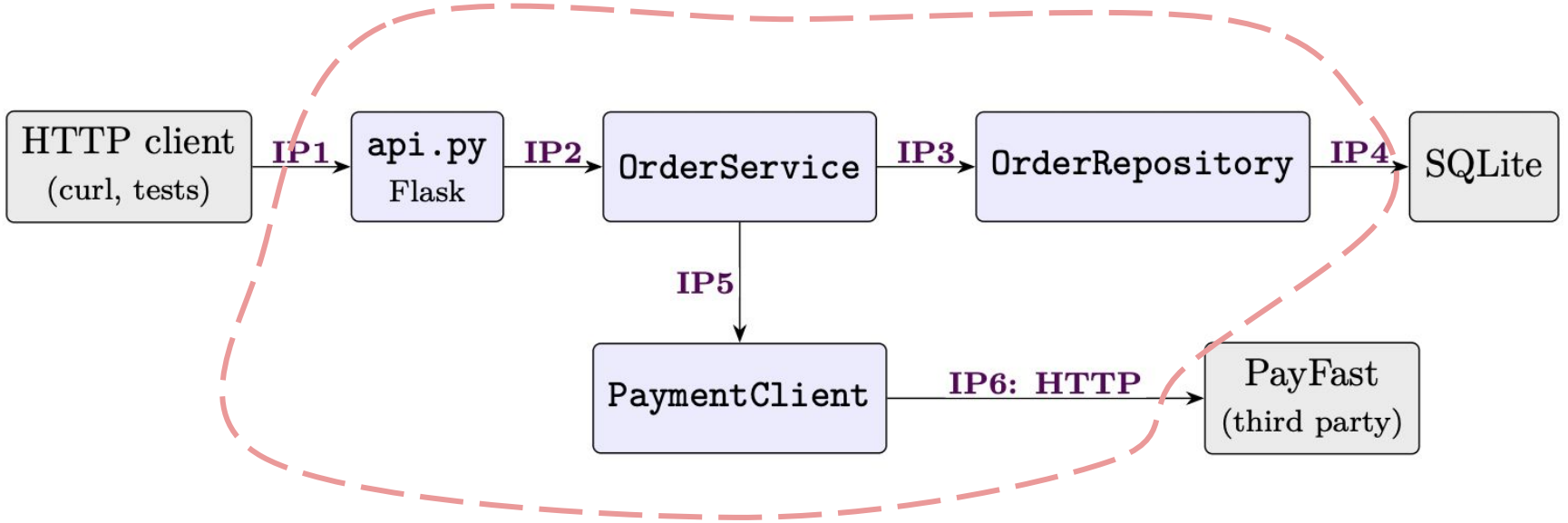
README.md

Setup

```
cd demos/integration_testing
pip install -r ../../requirements.txt
pytest
```

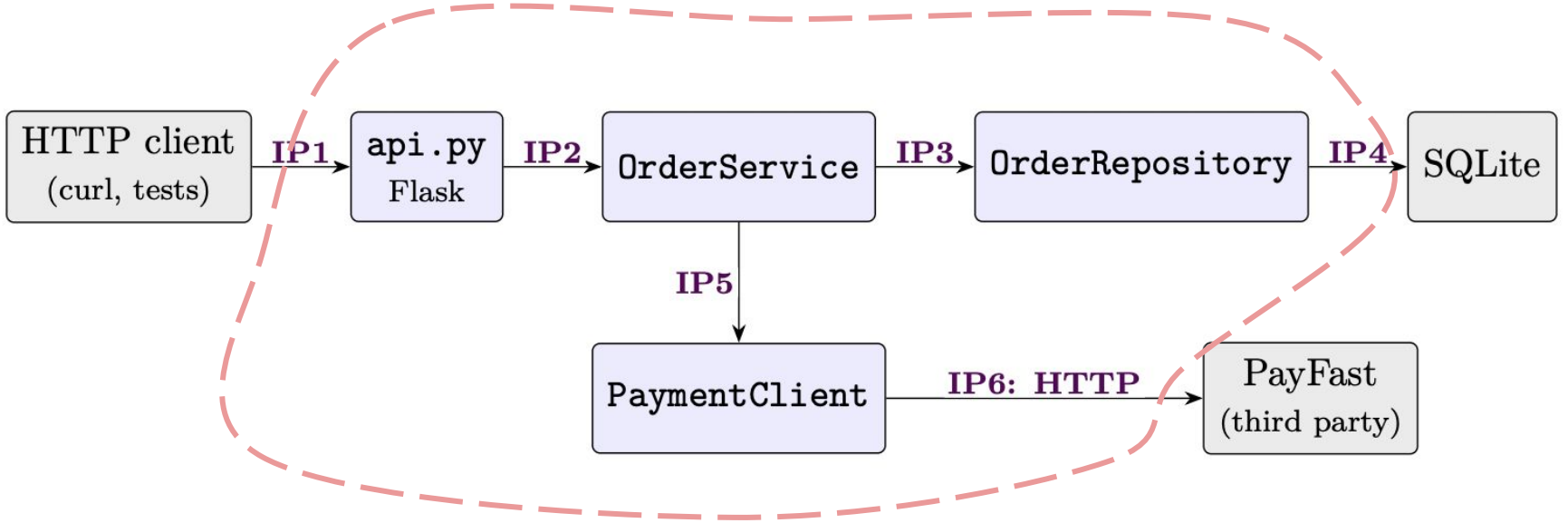
```
46 passed in 0.90s
```

Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

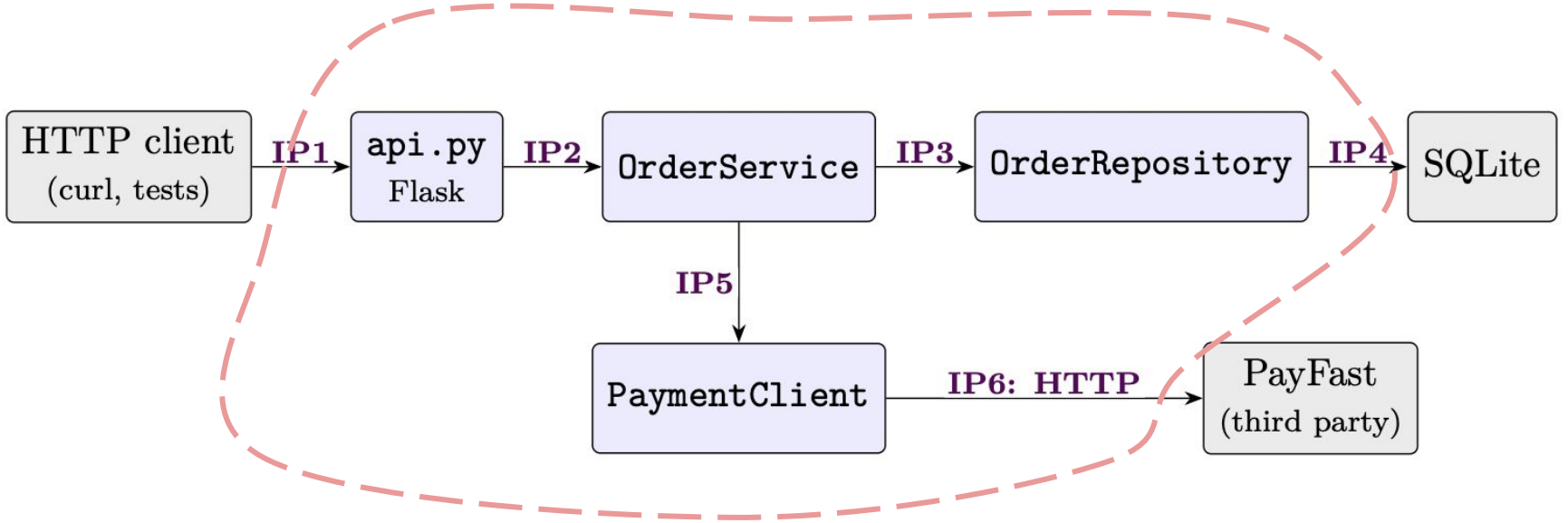
Detailed Integration Testing: Through a Practical Shop Application



The shop. Grey boxes are outside the shop's own code. The **Backend.** Each arrow is an integration point: a place where two components must agree.

ID	From	To	Kind	Contract
IP1	HTTP client	`api.py`	HTTP/JSON, API key	`specs/shop_api.md`
IP2	`api.py`	`OrderService`	Python call + exceptions	`service.py`, `errors.py`
IP3	`OrderService`	`OrderRepository`	Python call + transactions	`repository.py`
IP4	`OrderRepository`	SQLite	SQL	schema in `db.py`
IP5	`OrderService`	`PaymentClient`	Python call + exceptions	`payments.py`, `errors.py`
IP6	`PaymentClient`	PayFast	HTTP/JSON, Bearer token	`specs/payment_provider_api.md`

The shop: six integration points



IP	From → to	Over	Contract written in
IP1	HTTP client → api.py	HTTP/JSON, X-API-Key	`specs/shop_api.md`
IP2	api.py → OrderService	Python call + exceptions	`service.py, errors.py`
IP3	OrderService → OrderRepository	Python call + transactions	`repository.py`
IP4	OrderRepository → SQLite	SQL	`db.py (schema)`
IP5	OrderService → PaymentClient	Python call + exceptions	`payments.py, errors.py`
IP6	PaymentClient → PayFast	HTTP/JSON, Bearer token	`specs/payment_provider_api.md`

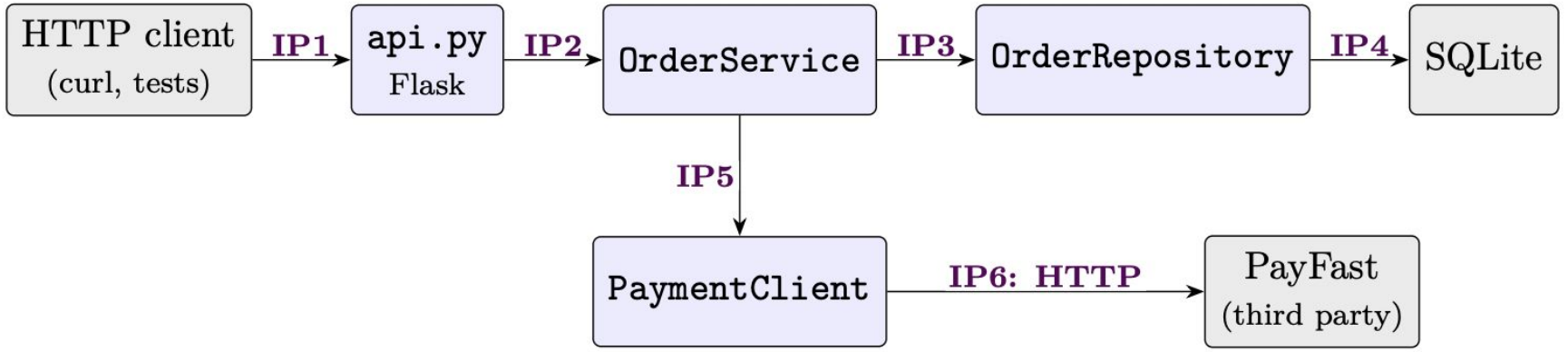
Review & Specification of the Shop App

The shop's review was held before its integration tests were written, on the architecture sketch, the two API specifications, and the component signatures. Its record is in `specs/design_review.md`. For example, PayFast's specification says `amount` is an integer number of *cents*, and notes that v1 took a decimal string of dollars. The reviewer noticed that the shop's payment client was written before v2. That became finding R1. The full set:

#	Finding	IP	Became tests in
R1	PayFast v2 requires integer cents; our client predates v2.	IP6	`test_payments_*`
R2	Nothing checks that the provider receives the same total we store.	IP3,5,6	`test_order_service`
R3	Reserving stock and creating the order must be atomic.	IP3,4	`test_repository`, `test_order_service`
R4	The charge cannot be inside the DB transaction. If it fails, who releases the stock?	IP3,5	`test_order_service`, `test_api`
R5	The provider's 5xx bodies are not guaranteed to be JSON.	IP6	`test_payments_*`
R6	The client retries a `POST` that moves money. Is that safe?	IP6	`test_payments_*`
R7	The first design had no timeout on the provider call.	IP6	`test_payments_live`
R8	Two credentials. A wrong *provider* key is our fault: 502, not "retry later".	IP1,6	`test_api*`, `test_payments_live`
R9	Every service exception needs a documented status; Flask's default errors are HTML.	IP1,2	`test_api_topdown`
R10	SQLite ignores foreign keys unless each connection enables them.	IP4	`test_repository`
R11	Where is input validated? Can bad input reach the service?	IP1,2	`test_api_topdown`

What is in the demo

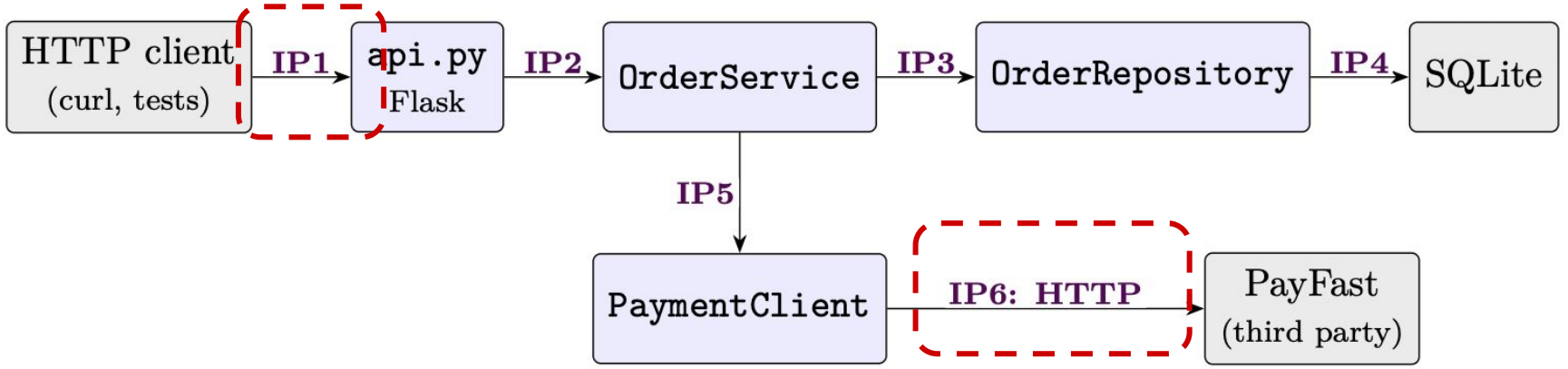
LIVE DEMO 2
setup + look around



Test file	What is real	Strategy step
`test_order_service_unit.py`	only the service (all mocked)	unit baseline
`test_repository.py`	repository + SQLite	bottom-up
`test_payments_stubbed.py`	client; provider stubbed	stub at HTTP
`test_payments_live.py`	client + fake provider	bottom-up
`test_order_service.py`	service + DB + fake provider	bottom-up 2
`test_api_topdown.py`	API; service stubbed	top-down
`test_api.py`	everything	assembled

What is in the demo

LIVE DEMO 2
setup + look around



API Integration Testing

Test file	What is real	Strategy step
`test_order_service_unit.py`	only the service (all mocked)	unit baseline
`test_repository.py`	repository + SQLite	bottom-up
`test_payments_stubbed.py`	client; provider stubbed	stub at HTTP
`test_payments_live.py`	client + fake provider	bottom-up
`test_order_service.py`	service + DB + fake provider	bottom-up 2
`test_api_topdown.py`	API; service stubbed	top-down
`test_api.py`	everything	assembled

The API by hand

LIVE DEMO 3
serve.py + curl

```
python serve.py        # shop on :8000, fake PayFast on :8001

curl -i -X POST http://127.0.0.1:8000/orders -H 'X-API-Key: shop-key' \
  -H 'Content-Type: application/json' \
  -d '{"sku": "MUG", "quantity": 2, "customer_email": "ana@example.com"}'
```

```
HTTP/1.1 201 CREATED
Content-Type: application/json
Location: /orders/1

{"charge_id":"ch_1","customer_email":"ana@example.com",
 "id":1,"quantity":2,"sku":"MUG","status":"paid",
 "total_cents":2500}
```

Every error is JSON: `{"error": "missing or invalid API key"}` A client that calls response.json() on an HTML error page crashes far from the real cause.

Now change one thing

Change	Answer
drop X-API-Key	401
`"quantity": "2"`	400
`"sku": "TEE"`	409
`--provider-mode decline`	402
`--provider-mode unavailable`	503

503 also sends Retry-After: 30

Top-down: the real API on a stub

LIVE DEMO 5
14 passed, no database

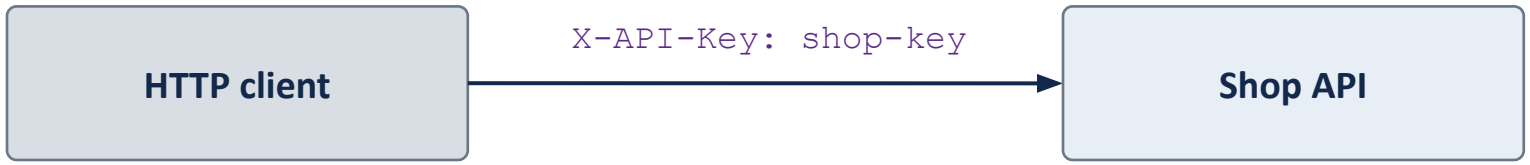
```
@pytest.mark.parametrize("raised, status", [
    pytest.param(UnknownProduct("MUG"), 404, id="unknown-sku"),
    pytest.param(OutOfStock("MUG"), 409, id="out-of-stock"),
    pytest.param(PaymentDeclined("card_declined"), 402, id="declined"),
    pytest.param(PaymentUnavailable("..."), 503, id="provider-down"),
    pytest.param(PaymentError("...: 401"), 502, id="our-bug"),
])
def test_service_errors_map_to_the_documented_status(raised, status):
    response = api(StubOrderService(raises=raised)).post(
        "/orders", json=GOOD_ORDER, headers=HEADERS)
    assert response.status_code == status
    assert response.is_json and "error" in response.get_json()
```

```
$ pytest tests/test_api_topdown.py -v
...status[unknown-sku]     PASSED
...status[out-of-stock]    PASSED
...status[declined]        PASSED
...status[provider-down]   PASSED
...status[our-bug]         PASSED
...never_reach_the_service[not-an-object] PASSED
...
14 passed in 0.12s
```

The stub records its calls: `assert stub.calls == []` proves bad input never reached the service.

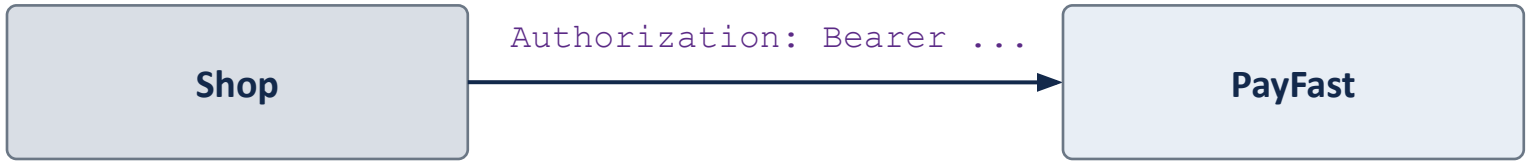
Two credentials, two different failures

Inbound (IP1)



Missing or wrong key:
401, before anything else runs

Outbound (IP6)



PayFast says 401: the shop is misconfigured.
502, no retry, not 503

Why not 503? 503 says "try again in 30 seconds". With the wrong key, every retry fails the same way. The right status is the one that tells the client what will happen next.

Try it: write two API tests

TRY IT 3
students · 10 min

1 In a new tests/test_mine.py, GET /orders/99 through the stubbed API. Assert the status and that the body is JSON.

```
from tests.test_api_topdown import (
    StubOrderService, api, HEADERS, GOOD_ORDER)

def test_unknown_order_is_json_404():
    r = api(StubOrderService()).get(
        "/orders/99", headers=HEADERS)
    assert ...
```

2 POST /orders with no body at all. What status and message? Prove the service was never called.

```
stub = StubOrderService()
r = api(stub).post("/orders", headers=HEADERS)
assert ...
assert stub.calls == []
```

Done early? Send "quantity": true. What does the API answer, and why? (Hint: Python's bool and int.)

Practice question

TRY IT
think, then vote · 2 min

With the outdated client switched in, `test_api.py` fails with `assert 502 == 201`.

Which single run points straight at the faulty integration point?

A `pytest tests/test_api_topdown.py`

B `SHOP_PAYMENTS=v1 pytest tests/test_payments_stubbed.py`

C `pytest tests/test_repository.py`

D `SHOP_PAYMENTS=v1 pytest tests/test_order_service_unit.py`

Answer: B

LIVE DEMO 6
check the answer

B `SHOP_PAYMENTS=v1 pytest tests/test_payments_stubbed.py → {'amount': '12.50'} != {'amount': 1250}`

\+ **Why** The contract check compares the request itself with PayFast's spec: integer cents, Bearer auth, Idempotency-Key. It names the field and both values.

× **A** The service is a stub: the payment client never runs.

× **C** The repository never touches payments.

× **D** Mocks pass with the bug: 3 passed.

Takeaways, and next time

1

Unit tests check the parts

Integration tests check the promises between them: formats, errors, credentials, state.

2

Integrate by risk

Incrementally, with stubs below and drivers above, so a failure points at what you just added.

3

Review, then test

Find integration risks in the design and the specs; turn each into a test, a change, or a decision.

Next time (Part 2): databases, external services, test doubles that check vs. lie, and automating integration tests.

Before then: read Sections 8-11; run python serve.py --provider-mode decline, then unavailable.

QA